

IDOR Lab - Student Grade System

Hệ thống học tập về lỗ hổng IDOR (Insecure Direct Object Reference) thông qua một ứng dụng quản lý điểm sinh viên được xây dựng bằng PHP và MySQL.

Mô tả

Đây là một lab thực hành về bảo mật web, mô phỏng hệ thống tra cứu điểm của sinh viên với lỗ hổng IDOR có thể cấu hình. Sinh viên có thể khai thác lỗ hổng để xem điểm của người khác hoặc truy cập vào các lớp học mà họ không được phép.

Mục tiêu học tập

- Hiểu được cơ chế hoạt động của lỗ hổng IDOR
- Biết cách phát hiện và khai thác IDOR trong ứng dụng web
- Nhận thức về tầm quan trọng của việc kiểm soát truy cập (Access Control)

Yêu cầu hệ thống

- Docker
- Docker Compose
- Web Browser

Cài đặt và Khởi chạy

Khởi động lab - Tự động gen random FLAG

Cách đơn giản nhất (Tự động 100%)

Với EMAIL tùy chỉnh (Khuyến nghị):

```
cd idor-lab
ENV=student_id EMAIL=long@mail.com docker-compose up -d
```

Hoặc để mặc định:

```
ENV=student_id docker-compose up -d
```

Chỉ vậy thôi! Mỗi lần chạy lệnh trên: - Tự động gen random điểm số, nhận xét cho 2.000 sinh viên (40 bạn/lớp) - Tự động random vị trí FLAG (Student ID hoặc Class ID)

- 50 lớp học với mã lớp random trong khoảng 100-500 - **FLAG được tạo bằng SHA1: FLAG{SHA1(DDMMYYYY_email_IDOR)}** - Ngày tháng theo GMT+7 (Giờ HCM) - Ví dụ: 24112025_long@mail.com_IDOR → SHA1 hash - Khởi động toàn bộ hệ thống

Muốn gen lại data mới với EMAIL khác:

```
docker-compose down -v # Xóa data cũ
ENV=class_id EMAIL=another@mail.com docker-compose up -d # Gen data mới
```

Công thức FLAG

FLAG được tạo bằng SHA1 hash theo công thức:

FLAG{SHA1(DDMMYYYY_email_IDOR)}

Ví dụ: - Date: 24112025 (24/11/2025 - GMT+7) - Email: long@mail.com - Input: 24112025_long@mail.com_IDOR - Output: FLAG{b22f5db2bcba00ea4cea3010136b5faa766944e2}

2. Kiểm tra trạng thái

```
docker-compose ps
```

Kết quả mong đợi:

NAME	STATUS
idor-lab-web-1	Up
idor-lab-db-1	Up

3. Truy cập ứng dụng

Mở trình duyệt và truy cập: **http://localhost:8080**

Tài khoản đăng nhập

Tài khoản thử nghiệm:

Username	Password	Class ID (random)	Role
alice	alice123	Tự động gán (40 SV/lớp)	Student
bob	bob123	Tự động gán (40 SV/lớp)	Student
student_1	password	Tự động gán (40 SV/lớp)	Student
student_2	password	Tự động gán (40 SV/lớp)	Student
admin	admin123	Tự động gán (40 SV/lớp)	Admin

Hướng dẫn khai thác IDOR

Cấu hình lỗ hổng

Lỗ hổng được điều khiển qua biến môi trường ENV. Bạn có 2 cách:

Cách 1: Truyền qua command line (Khuyến nghị)

```
ENV=student_id docker-compose up -d
# hoặc
ENV=class_id docker-compose up -d
```

Cách 2: Sửa file docker-compose.yml

```
environment:  
  - ENV=class_id # Thay đổi giá trị này
```

Sau đó: `docker-compose up -d`

Kịch bản 1: ENV=student_id

Mục tiêu: Tìm và xem điểm của sinh viên có FLAG

1. Khởi động lab với ENV=student_id
2. Kiểm tra file FLAG_INFO.json để biết student ID chứa FLAG (luôn nằm trong khoảng 1-40 vì cả lớp có 40 sinh viên)
3. Đăng nhập bằng tài khoản `alice` / `alice123`
4. Sau khi đăng nhập, URL sẽ có dạng:
`http://localhost:8080/grades.php?student_id=2&class_id=<class_cua_ban>&semester_id=SEM1`
5. Thay đổi `student_id=2` thành student ID trong FLAG_INFO.json (ví dụ `student_id=42`)
6. Bạn sẽ thấy môn “Secret Subject” với FLAG

FLAG Format: FLAG{STUDENT_IDOR_SUCCESS_ID_XX} (XX là student ID random)

Kịch bản 2: ENV=class_id

Mục tiêu: Tìm và truy cập lớp học bí mật của Alice

1. Khởi động lab với ENV=class_id:
`ENV=class_id docker-compose up -d`
2. Kiểm tra file FLAG_INFO.json để biết class ID chứa FLAG (50 lớp với mã random từ 100-500)
3. Đăng nhập bằng `alice` / `alice123`
4. URL ban đầu:
`http://localhost:8080/grades.php?student_id=2&class_id=<class_cua_ban>&semester_id=SEM1`
5. Thay đổi `class_id=<class_cua_ban>` thành class ID trong FLAG_INFO.json
6. Bạn sẽ thấy môn “Classified Training” với FLAG

FLAG Format: FLAG{CLASS_IDOR_SUCCESS_CLASS_XXX} (XXX là class ID random từ 100-500)

Dữ liệu hệ thống

- **Số lượng sinh viên:** 2.000 sinh viên
- **Số lượng lớp:** 50 lớp (mã lớp random trong khoảng 100-500)
- **Mỗi lớp:** 40 sinh viên cố định
- **Số môn học:** 12 môn
 - Mathematics, Physics, Chemistry, Biology
 - History, Literature, English, Computer Science
 - Geography, Economics, Physical Education, Art
- **Điểm mỗi sinh viên:** 5-7 môn học với nhận xét từ giáo viên
- **FLAG Locations:** Random mỗi lần chạy `generate_sql.py`
 - Xem file `FLAG_INFO.json` để biết vị trí hiện tại

Cấu trúc thư mục

```
idor-lab/
├── docker-compose.yml    # Cấu hình Docker Compose
├── Dockerfile            # Build PHP với MySQL extensions
├── generate_sql.py       # Script tạo dữ liệu mẫu
├── sql/
│   └── init.sql          # Database initialization
├── src/
│   ├── db.php            # Database connection
│   ├── index.php         # Login page
│   ├── grades.php        # Grade report (VULNERABLE)
│   └── logout.php        # Logout
```

Quản lý hệ thống

Xem logs

```
# Logs của web server
docker-compose logs web
```

```
# Logs của database
docker-compose logs db
```

```
# Theo dõi logs real-time
docker-compose logs -f
```

Tạo lại dữ liệu

```
# Tạo lại file SQL với dữ liệu mới
python3 generate_sql.py
```

```
# Khởi động lại với database mới
```

```
docker-compose down -v
docker-compose up -d
```

Dừng hệ thống

```
# Dừng containers
```

```
docker-compose down
```

```
# Dừng và xóa tất cả dữ liệu (bao gồm database)
```

```
docker-compose down -v
```

Cách phòng chống IDOR

Giải pháp đúng:

1. Luôn kiểm tra quyền truy cập:

```
if ($req_student_id != $_SESSION['user_id']) {
    die("Access Denied");
}
```

2. Kiểm tra enrollment:

```
$stmt = $pdo->prepare("SELECT * FROM enrollments WHERE student_id = ? AND class_id = ?");
$stmt->execute([$_SESSION['user_id'], $req_class_id]);
if (!$stmt->fetch()) {
    die("Not enrolled in this class");
}
```

3. Sử dụng session thay vì tham số URL:

- Lấy thông tin từ \$_SESSION thay vì \$_GET
- Không cho phép user tự chỉ định ID

4. Implement RBAC (Role-Based Access Control):

- Kiểm tra role/permission trước khi truy cập dữ liệu
- Sử dụng access control lists (ACL)

Tips

- Dùng Developer Tools (F12) để xem và sửa các tham số URL
- Quan sát sự khác biệt giữa các trường hợp vulnerable và secure
- Thử nghiệm với nhiều giá trị khác nhau của ID
- Đọc source code trong `src/grades.php` để hiểu logic kiểm tra

Hỗ trợ

Nếu gặp lỗi: 1. Kiểm tra Docker có đang chạy không 2. Xem logs: `docker-compose logs` 3. Khởi động lại: `docker-compose restart` 4. Reset

toàn bộ: `docker-compose down -v && docker-compose up -d`

Lưu ý

- Đây là LAB HỌC TẬP, KHÔNG sử dụng cho môi trường production
- Passwords được lưu dưới dạng plaintext (không bảo mật)
- Mục đích duy nhất là học tập về bảo mật web

License

MIT License - Dự án mã nguồn mở dành cho mục đích học tập.

Happy Hacking!